

PROCEDIMENTO OPERACIONAL PADRÃO PARA EXTRAÇÃO E ANÁLISE DE DADOS DE TAREFAS ADMINISTRATIVAS EM UM BANCO DE DADOS RELACIONAL

Gustavo Vieira Vale

Universidade Federal de Goiás

Estrada Municipal, Quadra e Área Lote 04 - Bairro Fazenda Santo Antônio, Aparecida de Goiânia - GO, CEP: 74971-451

vale@discente.ufg.br

RESUMO

Este Procedimento Operacional Padrão (POP) tem como objetivo orientar a extração e análise de dados de tarefas administrativas armazenadas em um banco de dados relacional. O trabalho considera o contexto de um escritório de médio porte, no qual um Departamento de Processos utiliza dados sobre setores, funcionários, processos, atividades, tarefas, prazos, status e prioridades para apoiar diagnósticos e melhorias internas. A metodologia proposta apresenta um passo a passo estruturado, contemplando a definição do objetivo da análise, delimitação do período analisado, conexão ao banco de dados, validação da estrutura, exploração dos dados, extração de dados consolidados, geração de indicadores, tratamento das informações, visualização dos resultados e proposição de ações de melhoria. Como resultado esperado, o POP permite transformar registros operacionais em informações gerenciais, possibilitando a identificação de gargalos, atrasos, acúmulo de demandas, produtividade por funcionário e oportunidades de melhoria nos processos administrativos.

PALAVRAS-CHAVE. Banco de Dados Relacional; Tarefas Administrativas; Melhoria de Processos.

Dados – Indicadores– Análise

ABSTRACT

This Standard Operating Procedure (SOP) aims to guide the extraction and analysis of administrative task data stored in a relational database. The study considers the context of a medium-sized office, in which a Process Department uses data on departments, employees, processes, activities, tasks, deadlines, status, and priorities to support internal diagnostics and process improvement. The proposed methodology presents a structured step-by-step procedure, including the definition of the analysis objective, delimitation of the analyzed period, database connection, structure validation, data exploration, consolidated data extraction, indicator generation, data treatment, result visualization, and proposal of improvement actions. As an expected result, the SOP enables the transformation of operational records into managerial information, supporting the identification of bottlenecks, delays, backlog, employee productivity, and improvement opportunities in administrative processes.

KEYWORDS. Relational Database; Administrative Tasks; Process Improvement.

Data – Indicators – Analysis

SUMÁRIO

1. Introdução	2
1.1. Visão Geral e Objetivos	2
1.2. Resumo Visual (fluxograma)	4
1.3. Glossário	4
2. Fundamentação Teórica	5
2.1. Sistemas de Informação Gerencial (SIG)	5
2.2. Arquitetura de Software	5
2.3. Banco de Dados: tipos de BD e sua evolução	6
2.4. Banco Relacional	6
2.5. Normalização	6
2.6. Relacionamentos de Entidades	7
2.7. Agregação e Composição	8
2.8. Conexão com Banco de Dados	8
3. Metodologia Proposta	9
3.1. Visão Geral do Procedimento	9
3.2. Estrutura do Banco de Dados	10
3.3. Ferramentas Utilizadas	14
3.4. Etapas do Procedimento	15
Etapa 1 – Definir Objetivo da Análise	15
Etapa 2 – Definir Período Analisado	16
Etapa 3 – Conectar ao Banco de Dados	16
Etapa 4 – Validar a Estrutura do Banco de Dados	17
Etapa 5 – Explorar os Dados	18
Etapa 6 – Extrair Dados Consolidados	19
Etapa 7 – Gerar Indicadores	21
Etapa 8 – Tratar e Organizar os Dados	22
Etapa 9 – Visualizar e Interpretar os Resultados	23
Etapa 10 – Propor Ações de Melhoria	24
4. Aplicação e Resultados	25
5. Considerações Finais	25
Referências	25

1. Introdução

1.1. Visão Geral e Objetivos

As organizações contemporâneas produzem grande volume de dados e informações em suas atividades rotineiras, incluindo registro de tarefas, prazos, responsáveis, setores etc. Em ambientes administrativos, esses dados costumam estar distribuídos em sistemas internos, planilhas ou banco de dados corporativos. De acordo com Laudon e Laudon (2023), os sistemas de informação possuem papel essencial na transformação de dados brutos em informações úteis para a gestão.

Nesse sentido, no contexto de um **escritório de médio porte, composto por diferentes setores e, aproximadamente, 60 funcionários**, a gestão das tarefas administrativas pode se tornar complexa quando não há instruções sobre como registrar, extrair e analisar os dados da organização. Essa falta de procedimento gera baixa rastreabilidade de informações, dificuldade de identificar gargalos e limitações na análise de desempenho dos processos internos. Assim, o **acesso direto ao banco de dados permite maior autonomia ao Departamento de Processos, possibilitando análises mais detalhadas** sobre carga de trabalho, atrasos, produtividade, backlog e retrabalho.

De acordo com Elmasri e Navathe [2018], a utilização de um Banco de Dados Relacional é adequada para esse cenário, pois permite organizar informações em tabelas conectadas entre si por meio de chaves primárias e estrangeiras. O modelo favorece a estruturação lógica dos dados, integridade de informações e consultas por meio da linguagem SQL. Dessa forma, é feita uma organização por meio de **entidades**, como setores, funcionários, processos e atividades, permitindo extração de indicadores para análise gerencial.

Além da estruturação das informações, a análise do banco de dados deve estar conectada à **Melhoria de Processos**, pilar fundamental para **diferenciar um Engenheiro de Produção de um Engenheiro de Software**. Portanto, a gestão de processos envolve, além da identificação e análise, o aprimoramento de processos organizacionais [Dumas et al., 2018].

Dessa forma, este trabalho propõe a elaboração de um Procedimento Operacional Padrão para orientar a conexão, extração, tratamento, análise e interpretação de dados de tarefas administrativas armazenadas em um banco de dados relacional com o intuito de construir um material consultável e claro, que permita compreender o passo a passo para transformar registros operacionais em informações gerenciais úteis à melhoria de processos administrativos.

Objetivo Geral:

- Elaborar um Procedimento Operacional Padrão para **extração e análise de dados de tarefas administrativas em banco de dados relacional**, visando apoiar o diagnóstico e a melhoria de processos em um escritório de médio porte.

Objetivos Específicos:

- Caracterizar a importância dos sistemas de informação e dos bancos de dados relacionais para a análise de processos administrativos;
- Propor uma estrutura lógica de banco de dados relacional para registro de tarefas, setores, funcionários, processos e atividades;
- Definir um procedimento padronizado para conexão, exploração e extração de dados;
- Planejar consultas SQL voltadas à geração de indicadores de desempenho administrativo;
- Estabelecer indicadores como volume de tarefas, backlog, taxa de atraso, tempo médio de execução, produtividade e taxa de retrabalho;
- Demonstrar como os dados extraídos podem apoiar a identificação de gargalos e a proposição de melhorias nos processos internos

1.2. Resumo Visual (fluxograma)

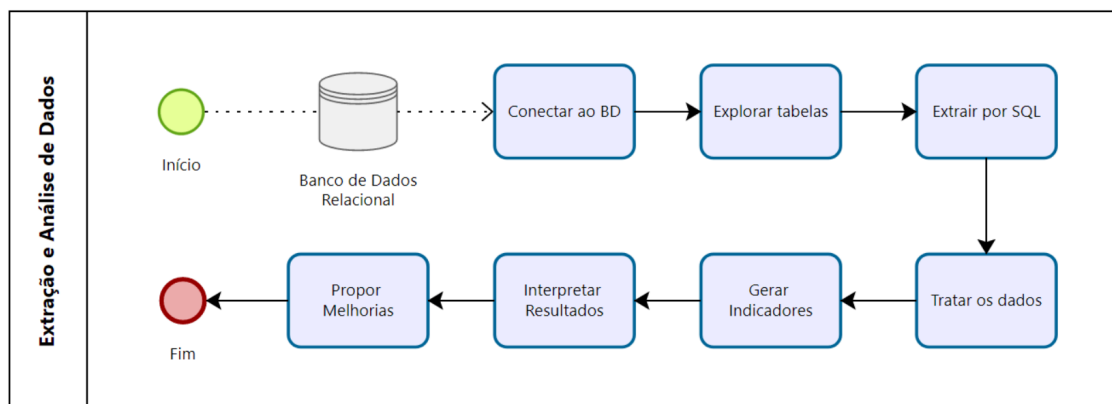


Figura 1 – Fluxo do POP para extração e análise de dados de tarefas administrativas.

1.3. Glossário

Neste artigo, haverá termos voltados à desenvolvedores de software, programadores e outros profissionais da área da Tecnologia da Informação, além de termos voltados a área de qualidade, produção e produtividade. Assim, foi elaborado um glossário (Tabela 1) para facilitar o entendimento completo de todo o conteúdo do artigo.

Termo	Definição
Banco de dados relacional	Estrutura de armazenamento de dados organizada em tabelas relacionadas entre si.
Tarefa administrativa	Registro de uma atividade executada por um funcionário ou setor dentro de um processo interno.
SQL	Linguagem utilizada para consultar, filtrar, agregar e manipular dados em bancos relacionais.
Chave primária	Campo que identifica de forma única cada registro de uma tabela.
Chave estrangeira	Campo que conecta uma tabela a outra, permitindo relacionamentos entre dados.
Backlog	Quantidade de tarefas ainda não concluídas em determinado período.
LeadTime	Tempo total entre a abertura e a conclusão de uma tarefa.
Taxa de atraso	Percentual de tarefas concluídas após o prazo previsto.
Retrabalho	Ocorrência em que uma tarefa precisa ser refeita, corrigida ou reaberta.

Indicador de desempenho	Medida utilizada para avaliar o comportamento de um processo ou atividade.
-------------------------	--

Tabela 1 – Relação entre termos e suas respectivas definições

2. Fundamentação Teórica

2.1. Sistemas de Informação Gerencial (SIG)

Os Sistemas de Informação Gerencial (SIG) são estruturas organizadas para coletar, processar, armazenar e disponibilizar **informações relevantes** para a tomada de decisões nas organizações [Laudon; Laudon, 2023]. Os sistemas de informação possuem a função de **transformar dados brutos em informações úteis**, permitindo que gestores acompanhem o desempenho organizacional, identificando problemas e tomando decisões com maior embasamento.

No contexto deste trabalho, os dados registrados sobre tarefas administrativas representam a “matéria-prima” do processo decisório. De forma isolada, esses registros são apenas dados operacionais. Entretanto, quando organizados e analisados, tornam-se informações gerenciais capazes de indicar gargalos, atrasos, sobrecarga de trabalho, baixa produtividade ou falha na execução dos processos internos.

Os Sistemas de Informação podem ser considerados cíclicos, pois é seguido um procedimento para cada uma das etapas do ciclo. São elas:

- Dados: registro bruto da tarefa;
- Informação: os dados transformados em indicadores;
- Conhecimento: interpretação das informações;
- Decisão: uso do conhecimento para propor melhorias.

Isso pode ser representado na Figura 2 abaixo:

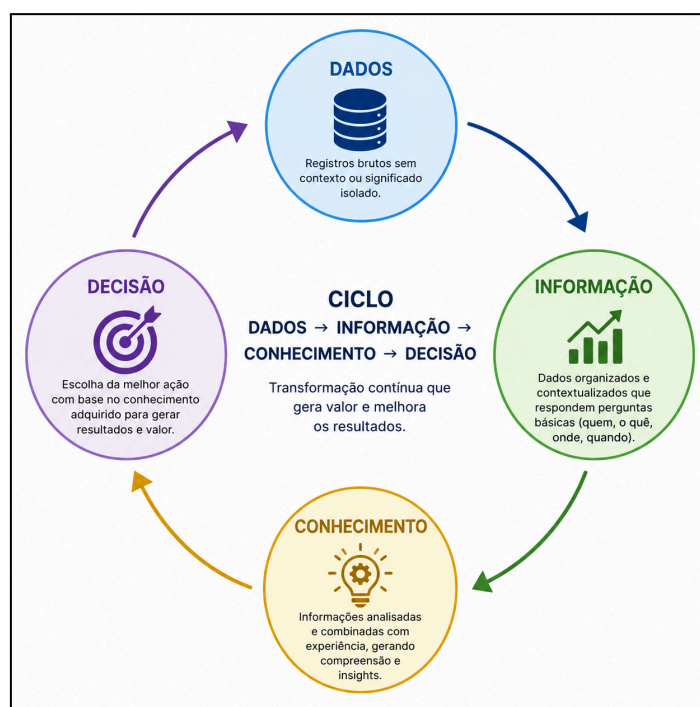


Figura 2 – Ciclo dos SIG

Dessa forma, o Departamento de Processos atua como usuário estratégico dos dados, utilizando o SIG como suporte para diagnóstico e melhoria dos processos administrativos.

2.2. Arquitetura de Software

A arquitetura de um software representa a forma como os componentes de um sistema são organizados e se relacionam entre si. Em sistemas corporativos, é comum que a aplicação seja dividida em camadas. Essa divisão, em sistemas de tarefas administrativas, costuma funcionar da seguinte maneira: **Interface do Usuário → Sistema de Tarefas → Regras de Negócio → Banco de Dados → Consultas e Indicadores.**

A interface é a parte com a qual os funcionários interagem para registrar, atualizar ou acompanhar tarefas. A camada de regras de negócio define critérios como prazo, prioridade, responsável, status e fluxo de aprovação. Já o banco de dados é responsável por armazenar os registros de forma estruturada e persistente.

Neste trabalho, a camada de banco de dados tem um papel crucial, pois, além dela ser uma camada desacoplada das demais partes, é nela que ficam armazenadas as informações necessárias para a análise dos processos administrativos.

2.3. Banco de Dados (tipos de BD e sua evolução)

Banco de dados é uma estrutura organizada para armazenar, gerenciar e recuperar dados de maneira eficiente. Os bancos de dados permitem representar informações do mundo real de forma estruturada, possibilitando consultas, atualizações, integridade e compartilhamento de dados entre diferentes usuários e aplicações [Elmasri; Navathe, 2018].

A evolução do banco de dados acompanhou o aumento da complexidade dos sistemas organizacionais. Inicialmente, os dados eram armazenados em arquivos sequenciais ou em estruturas hierárquicas. Posteriormente, os dados passaram a ser organizados em tabelas relacionadas entre si a partir do modelo relacional proposto por Edgar F. Codd, na década de 1970. Esse modelo se consolidou com a popularização da linguagem SQL e de sistemas como Oracle, PostgreSQL, MySQL e SQL Server.

A partir dos anos 2000, com o crescimento do volume, variedade e velocidade dos dados surgiram os bancos NoSQL, que são úteis em cenários que exigem maior flexibilidade, armazenamento de documentos, logs, sensores e aplicações distribuídas.

No caso deste trabalho, o **banco de dados relacional é a alternativa mais adequada**, pois as informações de um escritório podem ser naturalmente organizadas em entidades como setores, funcionários, processos, atividades e tarefas.

2.4. Banco Relacional

O banco de dados relacional organiza as informações em tabelas compostas por linhas e colunas. Cada linha representa um registro, enquanto que cada coluna representa um atributo. Segundo Elmasri e Navathe [2018], o modelo relacional é baseado na representação dos dados em relações entre si, o que permite estruturar as informações de maneira lógica e consistente. Esse modelo também permite a criação de vínculos entre tabelas por meio de chaves primárias e chaves estrangeiras.

Este tipo de estrutura permite realizar análises aprofundadas, como:

- Volume de tarefas por setor;
- Produtividade por funcionário;
- Tempo médio de execução;
- Taxa de Atraso;
- Taxa de Retrabalho.

2.5. Normalização

A normalização é um processo de **organizar os dados em tabelas para reduzir redundâncias e evitar inconsistências**. Significa compreender como as entidades se relacionam, garantindo a integridade entre as tabelas [Elmasri; Navathe, 2018]. A aplicação da normalização melhora a qualidade do banco e facilita a extração de indicadores.

As três principais formas normais são as mais importantes para este trabalho, apresentadas na Tabela 2:

Forma normal	Explicação	Exemplo no banco de tarefas
1FN	Cada campo deve conter apenas um valor atômico, sem listas ou grupos repetidos	Uma tarefa deve ter um único identificador de responsável principal, e não vários nomes no mesmo campo
2FN	Todos os atributos devem depender da chave primária completa	Em uma tabela de tarefas, a data de abertura depende da tarefa, não do funcionário
3FN	Atributos não-chave não devem depender de outros atributos não-chave	O nome do setor não deve ficar na tabela de tarefas; deve estar na tabela de setores

Tabela 2 – Formas de normalização

2.6. Relacionamentos de Entidades

Os relacionamentos de entidades representam as **conexões lógicas** entre as tabelas de um banco de dados. No caso de um banco de dados relacional, essas conexões são feitas por meio de **chaves primárias e estrangeiras**. A chave primária identifica, de forma única, cada registro de uma tabela, enquanto que a chave estrangeira conecta uma tabela a outra. Esses relacionamentos são fundamentais para representar corretamente a estrutura dos dados [Silberschatz; Korth; Sudarshan, 2019].

No cenário do escritório, os principais relacionamentos entre entidades podem ser representados de acordo com a Tabela 3:

Relacionamento	Interpretação	Exemplo no banco
1:1	Um funcionário pode ter um único usuário de acesso	Funcionário — Usuário
1:N	Um setor possui vários funcionários	Setor — Funcionários
1:N	Um processo possui várias atividades	Processo — Atividades
1:N	Uma atividade pode gerar várias tarefas	Atividade — Tarefas
1:N	Um funcionário pode ser responsável por várias tarefas	Funcionário — Tarefas

N:M	Uma tarefa pode envolver vários funcionários e um funcionário pode participar de várias tarefas	Tarefas — Participantes
-----	---	-------------------------

Tabela 3 – Exemplo de Relação entre Entidades para cada tipo de Relacionamento

A partir desta relação, há a possibilidade da criação de Diagramas de Relacionamento, uma ferramenta utilizada para melhorar a visualização das entidades e de seus relacionamentos. O Diagrama de Relacionamento com base na Tabela 3 está representado na Figura 2, no tópico 3.2.

2.7. Agregação e Composição

Agregação e composição são conceitos usados para representar **relações entre entidades**. Embora sejam mais comuns na modelagem orientada a objetos, também ajudam a compreender a dependência lógica entre elementos do banco de dados.

A **agregação** ocorre quando uma entidade está associada a outra, mas **ambas podem existir de forma independente**. No contexto deste trabalho, por exemplo, a relação entre *funcionários* e *tarefas* pode ser entendida como agregação: uma tarefa pode estar associada a um funcionário responsável, mas o funcionário continua existindo mesmo que uma tarefa seja excluída ou concluída.

A **composição**, por outro lado, ocorre quando **uma entidade depende da existência de outra**, por exemplo, a relação entre *tarefas* e *historico_tarefas*. O histórico só faz sentido quando a tarefa existir. Caso a tarefa seja removida, os registros de histórico relacionados a ela também perdem a finalidade. Relações de composição tendem a exigir maior controle de integridade, pois a exclusão ou alteração de uma entidade principal pode afetar diretamente os registros dependentes.

2.8. Conexão com Banco de Dados

A conexão com banco de dados é o processo pelo qual uma aplicação, ferramenta ou linguagem de programação acessa o banco para consultar, extrair ou manipular dados. Esse processo normalmente exige informações como endereço do banco de dados, porta, nome da base, usuário, senha e outras permissões de acesso.

Essa conexão permite que o Departamento de Processos, no contexto deste trabalho, acesse os dados diretamente na fonte, evitando que os dados passem por **“camadas intermediárias”** antes de chegar ao engenheiro, como relatórios prontos, planilhas exportadas e dashboards limitados. Alguns problemas desta abordagem é que **os dados já chegam filtrados, atrasados e agregados**. A partir dessa conexão, é possível executar consultas SQL, extrair dados para análise e gerar indicadores de desempenho administrativo.

Há diversas possibilidades de conexão, como PostgreSQL com *psycopg2*, MySQL com *mysql-connector*, MongoDB com *pymongo* e Cassandra com *cassandra-driver*. Para fins operacionais e contextos mais simples, o uso de SQLite pode ser adequado por sua praticidade, pois permite armazenar o banco em um único arquivo e executar consultas SQL sem necessidade de servidor. No entanto, PostgreSQL ou MySQL são opções mais robustas para armazenar dados de tarefas administrativas de um escritório, por exemplo.

Por fim, ressalta-se que a conexão ao banco também deve respeitar critérios de segurança e governança, como controle de acesso, permissões de leitura, proteção de dados sensíveis e anonimização quando necessário. Um complemento a isso é a Lei Geral de Proteção de Dados (LGPD), especialmente quando houver informações pessoais de funcionários. Portanto, este procedimento deve manter boas práticas de documentação, rastreabilidade e controle dos dados.

3. Metodologia Proposta

3.1. Visão Geral do Procedimento

A metodologia deste trabalho consiste na proposição de um Procedimento Operacional Padrão (POP) para orientar a **extração e análise de dados de tarefas administrativas armazenadas em um banco de dados relacional**. O procedimento foi estruturado com foco na atuação de um Departamento de Processos em um escritório de médio porte, composto por, aproximadamente, 60 funcionários distribuídos em diferentes setores administrativos.

O objetivo do POP é **padronizar o passo a passo** necessário para transformar registros operacionais de tarefas em informações gerenciais úteis à análise e melhoria de processos. Para isso, o procedimento contempla a definição do objetivo da análise, a proposição de ações de melhoria, passando pela conexão ao banco de dados, validação da estrutura, exploração dos dados, extração por meio de consultas SQL, geração de indicadores, tratamento das informações e interpretação dos resultados.

O escopo de aplicação do procedimento envolve tarefas administrativas executadas por setores, como **financeiro, comercial, marketing, recursos humanos, jurídico e operações**. Essas tarefas podem representar demandas internas, solicitações, atividades recorrentes, aprovações, análises, conferências, cadastros, atendimentos e entregas administrativas.

Por se tratar de um POP, a metodologia foi organizada de maneira **sequencial e reprodutível**. Assim, uma pessoa que consulte o material posteriormente deve conseguir compreender o que precisa ser feito, em qual ordem, com quais ferramentas e com qual finalidade.

Por fim, o cenário considerado pressupõe que o escritório possui um **sistema interno de gestão de tarefas**, no qual os funcionários registram suas atividades, responsáveis, datas, prazos, status e demais informações operacionais. Esses registros são armazenados em um banco de dados relacional, que funciona como fonte primária de informações. O Departamento de Processos, por sua vez, utiliza esse banco para **extrair dados, gerar indicadores e apoiar decisões** voltadas à melhoria dos fluxos administrativos. Além disso, pressupõe-se que o analista de dados tenha um conhecimento básico sobre banco de dados relacional, linguagem SQL, indicadores administrativos e melhoria de processos. Outros pré-requisitos estão dispostos na Tabela 4 a seguir:

Pré-requisito	Descrição
Banco de dados disponível	Deve existir uma base de dados relacional contendo registros de tarefas administrativas.
Estrutura mínima de tabelas	O banco deve conter tabelas relacionadas a setores, funcionários, processos, atividades e tarefas.
Permissão de acesso	O analista deve possuir autorização para acessar e consultar os dados.
Ferramenta de consulta	Deve haver uma ferramenta para conexão e execução de consultas SQL.
Definição do período analisado	O intervalo de tempo da análise deve ser previamente delimitado.
Objetivo da análise	Deve estar clara a pergunta gerencial que orientará a extração dos dados.

CrITÉrios de interpretação	Devem ser definidos critérios para atraso, backlog, retrabalho e produtividade.
Ambiente de documentação	O código, as queries, os resultados e as versões do trabalho devem ser documentados.

Tabela 4 – Pré-requisitos para a Execução do Procedimento

3.2. Estrutura do Banco de Dados

O banco de dados da empresa é um banco relacional, ou seja, **os dados são organizados em tabelas conectadas por meio de chaves primárias e chaves estrangeiras**. Essa estrutura permite evitar redundâncias, preservar a integridade das informações e facilitar consultas entre diferentes entidades, como setores, funcionários, processos, atividades e tarefas.

O banco de dados possui 8 tabelas principais que vão ser relevantes para o procedimento. São elas:

- **setores**: Armazena os setores existentes no escritório
- **funcionarios**: Armazena os dados dos 60 funcionários (incluindo a qual setor ele pertence)
- **processos**: Registra os processos administrativos executados pela empresa
- **atividades**: Representa as atividades que compõem cada processo
- **tarefas**: Registra cada tarefa executada ou aberta dentro do escritório
- **status_tarefa**: Registra a situação em que se encontra certa tarefa naquele momento
- **prioridades**: Nível de prioridade das tarefas de acordo com o Kanban
- **historico_tarefas**: Armazena alterações relevantes nas tarefas, como mudança de status, prazo ou responsável

De forma geral, a lógica dos relacionamentos pode ser representada da seguinte forma:

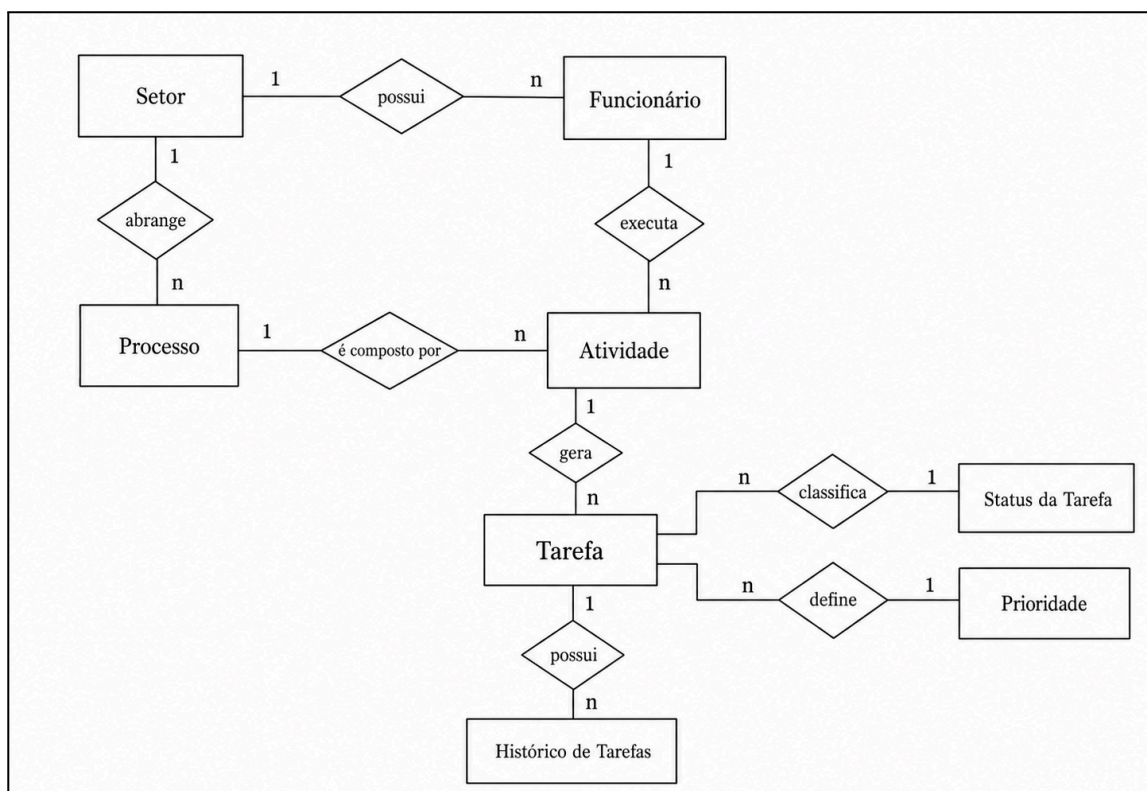


Figura 3 – Diagrama de Relacionamento do Banco de Dados

Essa estrutura significa que um setor pode possuir vários funcionários, mas cada funcionário pertence a um setor principal. Um setor também pode estar associado a vários processos administrativos. Cada processo pode ser dividido em diversas atividades, e cada atividade pode gerar várias tarefas ao longo do tempo. Cada tarefa, por sua vez, possui um responsável principal, um status, uma prioridade, podendo também estar associada a registros históricos.

A seguir, apresenta-se uma descrição simplificada dos principais campos de cada tabela.

Campo	Descrição
id_setor	Identificador único do setor.
nome_setor	Nome do setor administrativo.
descricao_setor	Breve descrição da função do setor.

Tabela 5 – Tabela **setores**

Campo	Descrição
id_funcionario	Identificador único do funcionário.
nome_funcionario	Nome do funcionário.
cargo	Cargo ocupado pelo funcionário.
id_setor	Identificador do setor ao qual o funcionário pertence.
data_admissao	Data de admissão do funcionário.
status_funcionario	Situação do funcionário, como ativo ou inativo.

Tabela 6 – Tabela **funcionarios**

Campo	Descrição
id_processo	Identificador único do processo.
nome_processo	Nome do processo administrativo.
id_setor	Setor responsável pelo processo.
descricao_processo	Descrição geral do processo.

Tabela 7 – Tabela **processos**

Campo	Descrição
--------------	------------------

id_atividade	Identificador único da atividade.
id_processo	Processo ao qual a atividade pertence.
nome_atividade	Nome da atividade executada.
descricao_atividade	Descrição da atividade.
tempo_padrao_estimado	Tempo previsto ou esperado para execução da atividade.

Tabela 8 – Tabela **atividades**

Campo	Descrição
id_tarefa	Identificador único da tarefa.
id_atividade	Atividade relacionada à tarefa.
id_responsavel	Funcionário responsável pela execução.
id_status	Status atual da tarefa.
id_prioridade	Prioridade atribuída à tarefa.
data_abertura	Data em que a tarefa foi registrada.
data_inicio	Data em que a execução foi iniciada.
data_prazo	Prazo previsto para conclusão.
data_conclusao	Data em que a tarefa foi concluída.
descricao_tarefa	Descrição resumida da tarefa.

Tabela 9 – Tabela **tarefas**

Campo	Descrição
id_status	Identificador único do status.
nome_status	Nome do status da tarefa.
descricao_status	Descrição do significado do status.

Tabela 10 – Tabela **status_tarefa**

Campo	Descrição
id_prioridade	Identificador único da prioridade.

nome_prioridade	Nível de prioridade da tarefa.
peso_prioridade	Valor numérico associado à prioridade.

Tabela 11 – Tabela prioridades

Campo	Descrição
id_historico	Identificador único do registro histórico.
id_tarefa	Tarefa relacionada ao histórico.
data_alteracao	Data da alteração realizada.
campo_alterado	Campo que foi modificado.
valor_anterior	Valor antes da alteração.
valor_novo	Valor após a alteração.
id_funcionario_alteracao	Funcionário responsável pela alteração.

Tabela 12 – Tabela historico_tarefas

A estrutura do banco de dados permite que o Departamento de Processos realize **análises em diferentes níveis**: por setor, por funcionário, por processo, por atividade, por prioridade e por status. Essa flexibilidade é importante porque a melhoria de processos depende da identificação precisa dos pontos críticos e não apenas de uma visão agregada da organização.

3.3. Ferramentas Utilizadas

Para a execução do POP, foram utilizadas ferramentas que permitem criar, consultar, extrair, analisar, visualizar e documentar os dados do banco relacional. A escolha das ferramentas considera a facilidade de uso, a compatibilidade com bancos relacionais, a possibilidade de reprodutibilidade e a adequação ao contexto acadêmico.

Finalidade	Ferramenta prevista	Justificativa
Banco de dados relacional	SQLite	Permite criar um banco local em arquivo único, sem necessidade de servidor.
Consulta visual ao banco	DB Browser for SQLite	Facilita a visualização das tabelas e execução de consultas SQL.
Linguagem de análise	Python	Permite conectar ao banco, extrair dados, tratar informações e gerar análises.
Bibliotecas de conexão e análise	sqlite3 e pandas	Permitem conectar ao SQLite e organizar os dados em DataFrames.

Visualização de dados	matplotlib ou planilha eletrônica	Permite criar gráficos para análise dos indicadores.
-----------------------	-----------------------------------	--

Tabela 13 – Finalidade e justificativa das ferramentas utilizadas

A ferramenta principal para o banco de dados é o **SQLite**, pois ele permite **construir uma base relacional simples e funcional** sem necessidade de configuração complexa. Isso facilita a execução do trabalho em que o foco está no procedimento de extração e análise dos dados, e não na administração de servidores.

Para a consulta visual, o uso do **DB Browser for SQLite** é a mais adequada, pois permite abrir o banco, visualizar as tabelas, verificar relacionamentos e executar comandos SQL. Essa parte é útil principalmente para validar se as tabelas foram criadas corretamente e se os dados estão disponíveis para análise.

Já para a extração e análise de dados, foi utilizado o **Python**, especialmente com as bibliotecas **sqlite3** e **pandas**. A biblioteca **sqlite3** permite estabelecer conexão direta com o banco SQLite, enquanto o **pandas** permite transformar o resultado das consultas SQL em tabelas analíticas, facilitando cálculos, filtros e geração de indicadores.

Por fim, para a visualização dos resultados, foram utilizados gráficos gerados em Python, com a biblioteca **matplotlib** especialmente, ou organizados em planilha eletrônica. A escolha dos gráficos dependerá da complexidade da análise. Para indicadores simples, um gráfico de barras é suficiente. Para análises temporais (como evolução mensal de tarefas abertas, evolução mensal de tarefas concluídas etc) é recomendado gráficos de linha e similares, por exemplo.

3.4. Etapas do Procedimento

As etapas a seguir descrevem o **passo a passo para executar o Procedimento Operacional Padrão de extração e análise de dados de tarefas administrativas**. O procedimento deve ser seguido em ordem, pois cada etapa gera informações ou validações necessárias para a etapa seguinte.

Etapa 1 – Definir Objetivo da Análise

A primeira etapa consiste em definir **qual pergunta gerencial será respondida** com a análise dos dados. Antes de acessar o banco, o usuário deve compreender qual problema deseja investigar, qual decisão pretende apoiar e quais indicadores serão necessários. Exemplos:

Necessidade gerencial	Pergunta de análise
Avaliar sobrecarga dos setores	Quais setores concentram maior volume de tarefas?
Identificar atrasos	Quais processos possuem maior taxa de atraso?
Avaliar produtividade	Quais funcionários concluíram mais tarefas no período?
Verificar acúmulo de demandas	Qual é o backlog por setor?
Investigar falhas de execução	Quais atividades apresentam maior taxa de retrabalho?

Tabela 14 – Necessidades e Perguntas Gerenciais

Após definir a pergunta, é necessário registrar o **objetivo da análise** em uma frase clara. Exemplo: “O objetivo da análise é identificar os setores com maior volume de tarefas em aberto e maior taxa de atraso, a fim de apoiar a priorização de melhorias nos processos administrativos”.

Além disso, devem ser definidos os **indicadores** que serão utilizados para responder ao objetivo, por exemplo:

Objetivo da análise	Indicadores recomendados
Identificar sobrecarga operacional	volume de tarefas, backlog, tarefas por funcionário
Avaliar cumprimento de prazos	taxa de atraso, tempo médio de execução
Identificar problemas de qualidade	taxa de retrabalho, tarefas reabertas
Avaliar desempenho por processo	tempo médio por processo, volume por processo, atraso por processo

Tabela 15 – Exemplo de Indicadores para cada Objetivo

Saída esperada da etapa: objetivo da análise registrado, pergunta gerencial definida e indicadores preliminares selecionados.

Etapa 2 – Definir Período Analisado

A segunda etapa consiste em **definir o intervalo de tempo** que será considerado na análise. Essa definição é necessária para evitar comparações incorretas e garantir que todos os indicadores sejam calculados com a mesma base temporária. Cada intervalo de tempo deve ter uma data inicial e uma data final.

Todas as consultas em SQL devem utilizar esse mesmo filtro e intervalo temporal. No banco de tarefas administrativas, por exemplo, recomenda-se utilizar o campo `data_abertura` como referência principal para selecionar as tarefas do período. A escolha do campo de data deve ser coerente com o indicador analisado, ex: Tarefas concluídas no período – `data_conclusao` –, Tarefas atrasadas – `data_prazo` e `data_conclusao`.

Saída esperada da etapa: período de análise definido e campo de data selecionado para filtrar os dados.

Etapa 3 – Conectar ao Banco de Dados

A terceira etapa consiste em estabelecer **conexão com o banco de dados relacional** que armazena os registros de tarefas administrativas. Neste POP, considera-se o uso de SQLite com Python por se tratar de uma alternativa simples e adequada.

Antes da conexão, é importante verificar se:

- o arquivo do banco está disponível (ex: `tarefas_administrativas.db`);
- está na mesma pasta do script ou com o caminho definido;
- as bibliotecas `sqlite3` e `pandas` estão disponíveis;
- o usuário possui permissão para acessar o arquivo;
- o nome do arquivo foi digitado corretamente.

Em seguida, a conexão será feita pelo código:


```
import sqlite3
import pandas as pd

conn = sqlite3.connect("tarefas_administrativas.db")
```

Código 1 – Conexão com o Banco de Dados Relacional em SQLite

Após a conexão, recomenda-se executar uma consulta simples para verificar se a conexão foi realizada corretamente:

```
teste_conexao = pd.read_sql_query("SELECT 1 AS conexao_ok;", conn)
print(teste_conexao)
```

Código 2 – Teste de Conexão

Se a conexão estiver correta, a saída do código deve ser, neste caso, “*conexao_ok*”. Caso ocorra algum erro, é necessário conferir o nome e o caminho do banco, instalar ou habilitar a biblioteca necessária, confirmar se as tabelas foram criadas no banco, revisar o nome do arquivo e o código e verificar o acesso à pasta ou ao arquivo.

Saída esperada da etapa: Conexão ativa com o Banco de Dados e a confirmação de acesso às tabelas.

Etapa 4 – Validar a Estrutura do Banco de Dados

A quarta etapa consiste em **verificar se o banco possui as tabelas e campos necessários para executar a análise**. Essa validação deve ser realizada antes de qualquer cálculo de indicador. Para consultar essas tabelas existentes, utiliza-se o código:

```
teste = pd.read_sql_query("SELECT name FROM sqlite_master WHERE type='table';", conn)
print(teste)
```

Código 3 – Execução de consulta SQL e armazenamento do resultado em DataFrame

Feito isso, é necessário **validar os campos principais** das tabelas centrais da análise. Neste caso, utilizou-se a tabela *tarefas*, no entanto, basta rodar o código a cada tabela que deseja validar. Segue o código abaixo:

```
informacoes_tabela = pd.read_sql_query("""PRAGMA table_info(tarefas);""", conn)
print(informacoes_tabela)
```

Código 4 – Validação dos Campos Principais

Com os campos validados, é possível verificar quais são os relacionamentos lógicos, por exemplo:

- *tarefas.id_responsavel* deve se relacionar com *funcionarios.id_funcionario*;
- *funcionarios.id_setor* deve se relacionar com *setores.id_setor*;
- *tarefas.id_atividade* deve se relacionar com *atividades.id_atividade*;
- *atividades.id_processo* deve se relacionar com *processos.id_processo*.

Saída esperada da etapa: confirmação de que o banco contém as tabelas, campos e relacionamentos necessários para análise.

Etapa 5 – Explorar os dados

A quinta etapa consiste em realizar **consultas iniciais para compreender o conteúdo do banco**. Essa etapa permite identificar volume de registros, tipos de dados, valores nulos, status existentes e possíveis inconsistências. Para verificar a quantidade total de tarefas, por exemplo, basta utilizar o código a seguir. Essa verificação é importante para entender o volume da base de dados.

```
total_tarefas = pd.read_sql_query("""SELECT COUNT(*) AS total_tarefas FROM tarefas;""", conn)
print(total_tarefas)
```

Código 5 – Quantidade Total de Tarefas

Após essa verificação, é possível visualizar os registros desejados, neste caso, 10 registros. O objetivo desta visualização é verificar se há padronização entre as tarefas registradas e entender qual seria esse padrão.

```
registros = pd.read_sql_query("""SELECT * FROM tarefas LIMIT 10;""", conn)
print(registros)
```

Código 6 – Visualização de registros

Além disso, também é possível verificar a distribuição dessas tarefas por status e por prioridade, utilizando os códigos a seguir, respectivamente. Utiliza-se essa verificação para avaliar uma nova padronização das tarefas.

```
distribuicao_status = pd.read_sql_query("""SELECT
    st.nome_status,
    COUNT(t.id_tarefa) AS total_tarefas
FROM tarefas t
JOIN status_tarefa st
    ON t.id_status = st.id_status
GROUP BY st.nome_status;""", conn)
print(distribuicao_status)
```

Código 7 – Distribuição de Tarefas por Status

```
distribuicao_prioridade = pd.read_sql_query("""SELECT
    p.nome_prioridade,
    COUNT(t.id_tarefa) AS total_tarefas
FROM tarefas t
JOIN prioridades p
    ON t.id_prioridade = p.id_prioridade
GROUP BY p.nome_prioridade;""", conn)
print(distribuicao_prioridade)
```

Código 8 – Distribuição de Tarefas por Prioridade

Por fim, para verificar se a qualidade dos dados está adequada ou se há uma quantidade considerável de falhas cadastrais, utiliza-se um código para avaliar a presença de campos nulos importantes:

```
campos_nulos = pd.read_sql_query("""SELECT
    COUNT(*) AS total_tarefas,
    COUNT(data_abertura) AS com_data_abertura,
    COUNT(data_prazo) AS com_data_prazo,
    COUNT(data_conclusao) AS com_data_conclusao
FROM tarefas;""", conn)

print(campos_nulos)
```

Código 9 – Presença de Campos Nulos

O objetivo desta etapa não é gerar conclusões finais, mas sim, conhecer a base a ser trabalhada antes de fazer análises consolidadas.

Saída esperada da etapa: entendimento inicial da base e identificação de possíveis problemas de qualidade dos dados

Etapa 6 – Extrair Dados Consolidados

A sexta etapa consiste em **executar consultas SQL** para consolidar os dados necessários da análise. Diferentemente da etapa anterior, que é exploratória, essa etapa busca gerar tabelas analíticas organizadas por setor, processo, funcionário, status ou período.

A primeira extração recomendada é o volume de tarefas por setor em determinado período com o objetivo de medir a carga de trabalho dos funcionários, por exemplo:

```
volume_tarefas = pd.read_sql_query("""SELECT
    s.nome_setor,
    COUNT(t.id_tarefa) AS total_tarefas
FROM tarefas t
JOIN funcionarios f
    ON t.id_responsavel = f.id_funcionario
JOIN setores s
    ON f.id_setor = s.id_setor
WHERE t.data_abertura BETWEEN '2026-01-01' AND '2026-03-31'
GROUP BY s.nome_setor
ORDER BY total_tarefas DESC;""", conn)

print(volume_tarefas)
```

Código 10 – Volume de Tarefas por Setor e por Período

Backlog por setor é outro fator importante para uma análise de dados em ambientes administrativos. Com ele, é possível verificar a quantidade de tarefas por setor que estão abertas naquele determinado período. Para isso, utiliza-se o seguinte código:

```
backlog = pd.read_sql_query("""SELECT
    s.nome_setor,
    COUNT(t.id_tarefa) AS tarefas_abertas
FROM tarefas t
JOIN funcionarios f
    ON t.id_responsavel = f.id_funcionario
JOIN setores s
    ON f.id_setor = s.id_setor
WHERE t.data_conclusao IS NULL
GROUP BY s.nome_setor
ORDER BY tarefas_abertas DESC;""", conn)

print(backlog)
```

Código 11 – Backlog por Setor

Outro dado importante é o tempo médio de execução por processo. Esse dado pode ser utilizado para verificar os processos que possuem uma maior demanda, e, consequentemente, um maior leadtime. Essa extração pode ser feita por meio do código:

```
tempo_de_execucao = pd.read_sql_query("""SELECT
    p.nome_processo,
    AVG(julianday(t.data_conclusao) - julianday(t.data_inicio)) AS tempo_medio_execucao
FROM tarefas t
JOIN atividades a
    ON t.id_atividade = a.id_atividade
JOIN processos p
    ON a.id_processo = p.id_processo
WHERE t.data_conclusao IS NOT NULL
GROUP BY p.nome_processo
ORDER BY tempo_medio_execucao DESC;""", conn)

print(tempo_de_execucao)
```

Código 12 – Tempo Médio de Execução por Processo

Por fim, para a Engenharia de Produção um dos dados mais importantes é o de produtividade por funcionário. Esse dado também deve ser extraído no ambiente administrativo e por meio do seguinte código abaixo:

```
produtividade_funcionario = pd.read_sql_query("""SELECT
    f.nome_funcionario,
    s.nome_setor,
    COUNT(t.id_tarefa) AS tarefas_concluidas
FROM tarefas t
JOIN funcionarios f
    ON t.id_responsavel = f.id_funcionario
JOIN setores s
    ON f.id_setor = s.id_setor
JOIN status_tarefa st
    ON t.id_status = st.id_status
WHERE st.nome_status = 'Concluída'
GROUP BY f.nome_funcionario, s.nome_setor
ORDER BY tarefas_concluidas DESC;""", conn)

print(produtividade_funcionario)
```

Código 13 – Produtividade por Funcionário

Feito as extrações desejadas, é necessário salvá-las com nomes claros, indicando a análise relacionada, como `query_volume_tarefas_setor.sql`.

Saída esperada da etapa: tabelas consolidadas com os dados necessários para cálculo e análise dos indicadores.

Etapa 7 – Gerar Indicadores

A sétima etapa consiste em **transformar os dados consolidados em indicadores de desempenho administrativo**. Cada indicador deve possuir definição, fórmula, fonte de dados e interpretação esperada. Alguns indicadores podem ser os próprios dados extraídos da etapa anterior, mas outros precisam ser calculados para alcançar a resposta da pergunta definida no início deste procedimento. Alguns indicadores são:

Indicador	Fórmula	Finalidade
Volume de tarefas por setor	total de tarefas do setor	medir carga de trabalho
Backlog	tarefas abertas ou não concluídas	medir acúmulo de demanda
Taxa de atraso	$\text{tarefas atrasadas} \div \text{total de tarefas} \times 100$	avaliar cumprimento de prazos
Tempo médio de execução	média entre início e conclusão	medir eficiência operacional
Produtividade por funcionário	tarefas concluídas por funcionário	avaliar capacidade de entrega
Taxa de retrabalho	$\text{tarefas com retrabalho} \div \text{total de tarefas} \times 100$	avaliar qualidade da execução

Tabela 16 – Exemplos de Indicadores

Esses cálculos podem ser feitos manualmente ou também, utilizando os códigos em Python. Por exemplo, para se calcular a taxa de atraso, utiliza-se o código:

```
taxa_atraso = pd.read_sql_query("""SELECT
    s.nome_setor,
    COUNT(CASE
        WHEN t.data_conclusao > t.data_prazo THEN 1
    END) * 100.0 / COUNT(t.id_tarefa) AS taxa_atraso
FROM tarefas t
JOIN funcionarios f
    ON t.id_responsavel = f.id_funcionario
JOIN setores s
    ON f.id_setor = s.id_setor
WHERE t.data_conclusao IS NOT NULL
GROUP BY s.nome_setor
ORDER BY taxa_atraso DESC;""", conn)

print(taxa_atraso)
```

Código 14 – Taxa de Atraso de Tarefas

Para cada indicador calculado, o usuário deve registrar o nome do indicador, seu objetivo, sua fórmula, a fonte utilizada, o intervalo analisado, o resultado e a interpretação.

Saída esperada da etapa: indicadores calculados e organizados para análise

Etapa 8 – Tratar e Organizar os Dados

A oitava etapa consiste em **preparar os dados extraídos para análise e apresentação**. Mesmo quando os dados são extraídos corretamente, pode ser necessário realizar reajustes para melhorar a leitura, corrigir inconsistências ou criar campos calculados. Em análises por Python, as **consultas SQL devem ser transformadas em DataFrames** e verificar os tipos de dados. Além disso, para os campos de data, é necessário converter os valores para o formato adequado.

```
df = pd.read_sql_query(query, conn) #Transformar em DataFrame

df.info() #Verificar os tipos de dados

#Converter os valores para o formato adequado
df["data_abertura"] = pd.to_datetime(df["data_abertura"])
df["data_conclusao"] = pd.to_datetime(df["data_conclusao"])
df["data_prazo"] = pd.to_datetime(df["data_prazo"])
```

Código 15 – Tratamento de Dados

Outro tratamento que pode ser necessário é a criação de colunas nos DataFrames, por exemplo, calcular o tempo de execução por meio da data de conclusão e da data de início:

```
df["tempo_execucao"] = (df["data_conclusao"] - df["data_inicio"]).dt.days
```

Código 16 – Exemplo para Criação de Coluna

Para a organização dos resultados, a sugestão é criar tabelas finais com nomes claros e colunas relevantes, como:

Tabela analítica	Conteúdo
df_tarefas_setor	volume de tarefas por setor
df_backlog	tarefas abertas por setor
df_atrasos	taxa de atraso por setor
df_produtividade	tarefas concluídas por funcionário
df_retrabalho	taxa de retrabalho por processo

Tabela 17 – Exemplo de Tabelas Analíticas

Durante o tratamento de dados, é necessário verificar se os valores nulos devem ser removidos, preenchidos ou mantidos, se as datas inconsistentes representam erros ou condições especiais, se é necessário padronizar nomenclaturas em status fora do padrão, e verificar se outliers são erros de sistema ou são casos reais a serem levados em consideração.

Saída esperada da etapa: dados tratados, organizados e prontos para visualização e interpretação.

Etapa 9 – Visualizar e Interpretar os Resultados

A penúltima etapa consiste em **transformar as tabelas de indicadores em visualizações e interpretações gerenciais**. O objetivo é facilitar a visualização e leitura dos resultados e apoiar a identificação de padrões, gargalos e oportunidades de melhorias.

Cada indicador deve ser apresentado com o tipo de visualização mais adequado, por exemplo, o gráfico de total de tarefas por setor pode ser visualizado com um gráfico de barras por meio do código:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.bar(
    df_tarefas_setor["nome_setor"],
    df_tarefas_setor["total_tarefas"]
)

plt.title("Volume de tarefas por setor")
plt.xlabel("Setor")
plt.ylabel("Total de tarefas")
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

Código 17 – Geração do Gráfico de Volume de Tarefas por Setor

Outro procedimento importante é seguir uma padronização para a interpretação de cada um dos gráficos e tabelas gerados. Isso evita análises superficiais e garante que o usuário relacione os dados com possíveis causas e impactos nos processos. Um modelo de interpretação pode ser:

```
print("Indicador analisado: taxa de atraso por setor")
print("Resultado encontrado: o setor financeiro apresentou a maior taxa de atraso no período analisado.")
print("Área/processo crítico: financeiro.")
print("Possível causa: concentração de tarefas em poucos responsáveis, excesso de aprovações manuais" \
      " ou prazos incompatíveis com a complexidade das demandas.")
print("Impacto no processo: aumento do lead time administrativo e maior risco de acúmulo de tarefas pendentes.")
print("Recomendação inicial: mapear o fluxo das atividades financeiras, revisar prazos" \
      " e avaliar redistribuição das tarefas entre os responsáveis.")
```

Código 18 – Modelo de Interpretação

A interpretação não deve se limitar a descrever o gráfico, mas sim, buscar explicar o que o resultado significa para o processo.

Saída esperada da etapa: gráficos, tabelas interpretadas e principais achados da análise

Etapa 10 – Propor Ações de Melhoria

A última etapa consiste em **transformar os achados da análise em ações práticas de melhoria**. Essa etapa conecta o uso do banco de dados com a atuação do Departamento de Processos. As ações devem ser propostas com base nos indicadores analisados, evitando recomendações genéricas sem relação com os dados.

A sugestão é estruturar as ações de melhoria em formato objetivo, como uma matriz 5W2H ou um Plano de Ação baseado em uma matriz de Problema x Solução.

Ao final das definições de melhorias, o usuário deve consolidar as recomendações em uma tabela final com um resumo do plano de ação, como:

Prioridade	Problema identificado	Evidência nos dados	Ação recomendada	Responsável sugerido
Alta	Backlog elevado	maior número de tarefas abertas	balancear demandas	gestor do setor
Alta	Atrasos recorrentes	maior taxa de atraso	revisar prazos e aprovações	processos + gestor
Média	Retrabalho elevado	maior taxa de retrabalho	criar checklist padrão	processos
Média	Tempo médio elevado	maior lead time	redesenhar fluxo	processos
Baixa	Baixa visibilidade	ausência de dashboards	criar painel periódico	processos/TI

Tabela 18 – Exemplo de um Resumo de um Plano de Ação

Saída esperada da etapa: plano inicial de ações de melhoria baseado nos indicadores extraídos do banco de dados.

4. Aplicação e Resultados

5. Considerações Finais

Referências

Bibliografias principais

Laudon, Kenneth C.; Laudon, Jane P. Sistemas de informação Gerenciais. São Paulo: Pearson, 2023.

Elmasri, Ramez; Navathe, Shamkant B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2018.

Bibliografias complementares

Dumas, Marlon; La Rosa, Marcello; Mendling, Jan; Reijers, Hajo A. Fundamentals of Business Process Management. 2. ed. Berlin: Springer, 2018.

Silberschatz, Abraham; Korth, Henry F.; Sudarshan, S. Database System Concepts. 7. ed. New York: McGraw-Hill, 2019.

Davenport, Thomas H.; Harris, Jeanne G. Competing on analytics: the new science of winning. Boston: Harvard Business School Press, 2007.